

ĐẠI HỌC BÁCH KHOA HÀ NỘI

Khoa/Trường Công nghệ Thông tin và Truyền thông



BÁO CÁO BÀI TẬP LỚN

Môn học: Nhập môn Trí tuệ nhân tạo

Hệ thống gợi ý sản phẩm thời trang H&M

So sánh Popularity, Matrix Factorization, Neural Collaborative Filtering
và Hybrid đa phương thức

Nhóm thực hiện:

Thành viên:

Giảng viên hướng dẫn:

Nhóm 9

Nguyễn Đăng Long - 202400057

Nguyễn Thế Khải - 202400050

Phạm Gia Linh - 202416262

TS. Ngô Văn Linh

Hà Nội, tháng 06 năm 2026

Mục lục

Tóm tắt	1
1 Giới thiệu	1
1.1 Bối cảnh	1
1.2 Mục tiêu của project	1
1.3 Phạm vi báo cáo	2
2 Tài liệu liên quan và cơ sở lựa chọn phương pháp	2
2.1 Hệ gợi ý với implicit feedback	2
2.2 Matrix Factorization và collaborative filtering	2
2.3 Neural Collaborative Filtering	3
2.4 Visual feature trong fashion recommendation	3
2.5 Đánh giá trong fashion recommendation	3
3 Mô tả bài toán	3
3.1 Đầu vào và đầu ra	3
3.2 Đặc điểm implicit feedback	4
3.3 Mô hình hóa bài toán xếp hạng	4
3.4 Thách thức cold-start và long-tail	4
4 Dữ liệu và tiền xử lý	5
4.1 Nguồn dữ liệu	5
4.2 Tiền xử lý và split theo thời gian	5
4.3 Lý do mã hóa ID và chuẩn hóa bảng dữ liệu	6
4.4 Kiểm soát leakage khi chia dữ liệu	6
4.5 Quy mô dữ liệu	6
4.6 Độ phủ train/test	7
4.7 Tính tập trung theo popularity	7
4.8 Visual feature	8
4.9 Quy trình trích xuất đặc trưng ảnh	8
5 Cơ sở lý thuyết	9
5.1 MAP@12	9
5.2 HitRate@12 và NDCG@12	9
5.3 Binary Cross Entropy và negative sampling	10
5.4 Pairwise ranking và listwise ranking	10
5.5 Sampled candidate và full-catalog retrieval	11

6	Các mô hình	11
6.1	Popularity baseline	11
6.2	Matrix Factorization	11
6.3	Neural Collaborative Filtering	12
6.4	Hybrid đa phương thức	13
6.5	So sánh mức độ thông tin của các mô hình	14
7	Pipeline thực nghiệm	15
7.1	Lệnh chạy chính	15
7.2	Luồng xây dựng mô hình trong code	16
7.3	Chi tiết negative sampling trong training	16
7.4	Batch training và tối ưu	17
7.5	Thiết lập huấn luyện	17
7.6	Best checkpoint và overfitting	17
7.7	Cách diễn giải kết quả offline	18
7.8	Biểu đồ training	18
8	Tái lập thực nghiệm và artifact kiểm chứng	19
8.1	Các artifact chính	19
8.2	Checklist tái lập	19
8.3	Các lệnh đã dùng	19
9	Kết quả thực nghiệm	20
9.1	Kết quả tổng thể	20
9.2	Đọc kết quả tổng thể	21
9.3	Phân tích Popularity	21
9.4	Phân tích MF	22
9.5	Phân tích NCF	22
9.6	Phân tích Hybrid	22
10	Phân tích theo segment	23
10.1	Theo lịch sử user	23
10.2	Theo độ phổ biến item	24
10.3	Theo visual availability	24
11	Thảo luận	24
11.1	Mức độ chắc chắn của các kết luận	24
11.2	Vì sao Popularity tốt nhất?	25

11.3 Vì sao Hybrid chưa thắng?	25
11.4 Bài học từ thực nghiệm	26
12 Hạn chế	26
13 Hướng phát triển	26
13.1 Candidate generation	26
13.2 Ranking stage	27
13.3 Cải tiến Hybrid	27
14 Kết luận	27

Danh sách hình vẽ

1	Mức tập trung popularity trong train và test	7
2	Độ phủ visual feature sau khi trích xuất đầy đủ	8
3	Train/validation loss trong quá trình huấn luyện	18
4	So sánh MAP@12, HitRate@12 và NDCG@12	20
5	Popularity trong sampled candidate và full-catalog filtered evaluation . .	22
6	MAP@12 theo nhóm độ phổ biến item	23
7	MAP@12 theo nhóm lịch sử user	23

Danh sách bảng

1	Các thành phần dữ liệu sử dụng	5
2	Khoảng thời gian train/test	5
3	Tỷ trọng tương tác của các nhóm item phổ biến	7
4	So sánh tín hiệu đầu vào của các mô hình	14
5	Ưu điểm và hạn chế của các mô hình trong phạm vi project	15
6	Artifact chính của thực nghiệm	19
7	Kết quả tổng thể trên tập test	20
8	MAP@12 theo nhóm lịch sử user	24
9	MAP@12 theo nhóm độ phổ biến item	24

Tóm tắt

Báo cáo này trình bày quá trình xây dựng và đánh giá một hệ thống gợi ý sản phẩm thời trang dựa trên dữ liệu H&M Personalized Fashion Recommendations. Bài toán được đặt dưới dạng gợi ý top-12 sản phẩm cho mỗi khách hàng trong tuần kiểm thử. Nhóm triển khai và so sánh bốn hướng tiếp cận: Popularity baseline, Matrix Factorization (MF), Neural Collaborative Filtering (NCF) và một mô hình Hybrid kết hợp collaborative filtering, metadata sản phẩm và visual feature từ ảnh sản phẩm.

Kết quả thực nghiệm trên bộ dữ liệu đã xử lý lại cho thấy thứ tự hiệu năng tổng thể là Popularity > MF > Hybrid > NCF. Popularity đạt $\text{MAP@12} = 0.06640$, MF đạt 0.06016, Hybrid đạt 0.05325 và NCF đạt 0.03997. Điều này cho thấy dữ liệu kiểm thử của H&M có tính xu hướng rất mạnh: một số sản phẩm phổ biến trong giai đoạn gần test có ảnh hưởng lớn tới kết quả. Mặc dù Hybrid chưa vượt MF và Popularity ở metric tổng thể, mô hình này vẫn thể hiện vai trò của tín hiệu nội dung, đặc biệt ở các nhóm item ít phổ biến hơn hoặc item có ít lịch sử tương tác.

1 Giới thiệu

1.1 Bối cảnh

Trong thương mại điện tử, hệ thống gợi ý giúp rút ngắn quá trình tìm kiếm sản phẩm và tăng khả năng người dùng tiếp cận đúng mặt hàng phù hợp. Với ngành thời trang, bài toán này khó hơn nhiều lĩnh vực khác do sản phẩm thay đổi nhanh theo mùa vụ, xu hướng xã hội, bộ sưu tập mới và khả năng xuất hiện các item có rất ít lịch sử mua.

Dữ liệu H&M Personalized Fashion Recommendations là một bộ dữ liệu thực tế gồm lịch sử giao dịch, thông tin khách hàng, thông tin sản phẩm và ảnh sản phẩm. Mục tiêu của bài toán là gợi ý danh sách 12 sản phẩm có khả năng được khách hàng mua trong tuần kế tiếp, tương ứng với thiết lập đánh giá MAP@12 của cuộc thi Kaggle [1]. Đây là bài toán implicit feedback vì dữ liệu chỉ cho biết các lượt mua đã xảy ra; các sản phẩm không mua không đồng nghĩa chắc chắn là người dùng không thích, mà thường được dùng làm negative sample trong quá trình huấn luyện [2, 3].

1.2 Mục tiêu của project

Project hướng tới ba mục tiêu chính:

1. Xây dựng pipeline dữ liệu hoàn chỉnh từ dữ liệu gốc tới dữ liệu huấn luyện, kiểm thử và đặc trưng ảnh.
2. Huấn luyện và so sánh các mô hình gợi ý đại diện cho nhiều mức độ phức tạp: Popularity, MF, NCF và Hybrid.

3. Phân tích kết quả một cách khách quan, không chỉ nhìn vào bảng metric tổng thể mà còn đánh giá theo các nhóm user/item để hiểu khi nào mô hình cá nhân hóa và tín hiệu nội dung có tác dụng.

1.3 Phạm vi báo cáo

Báo cáo tập trung vào phiên bản thực nghiệm mới nhất:

- Dữ liệu: `data\processed_new`.
- Visual feature: `visual_features_full.npz`.
- Checkpoint: `checkpoints\processed_new`.
- Kết quả: `reports\report_pipeline_processed_new`.

Các số liệu trong báo cáo được lấy từ pipeline thực nghiệm full profile đã chạy xong.

2 Tài liệu liên quan và cơ sở lựa chọn phương pháp

2.1 Hệ gợi ý với implicit feedback

Trong nhiều hệ thống thương mại điện tử, người dùng hiếm khi cung cấp rating tường minh. Thay vào đó, hệ thống quan sát hành vi như mua hàng, click, xem sản phẩm hoặc thêm vào giỏ hàng. Các tín hiệu này được gọi là implicit feedback. Hu, Koren và Volinsky chỉ ra rằng implicit feedback khác explicit rating ở chỗ nó thể hiện mức độ quan sát/hành vi hơn là mức độ yêu thích tuyệt đối; vì vậy khi mô hình hóa cần phân biệt giữa preference và confidence của tín hiệu [2]. Trong project này, lượt mua được coi là positive feedback, còn negative feedback được lấy mẫu từ những sản phẩm user chưa mua.

Rendle và cộng sự đề xuất BPR cho bài toán personalized ranking từ implicit feedback, nhấn mạnh rằng nhiều phương pháp học từ implicit feedback không tối ưu trực tiếp cho ranking nếu chỉ dùng objective dạng pointwise [3]. Điều này là nền tảng cho nhận định trong báo cáo: BCE loss giảm không đảm bảo MAP@12 tăng, vì BCE là objective phân loại cặp user-item còn MAP@12 là metric xếp hạng danh sách.

2.2 Matrix Factorization và collaborative filtering

Matrix Factorization là một họ mô hình nền tảng trong recommender systems, đặc biệt phổ biến sau Netflix Prize. Koren, Bell và Volinsky mô tả cách biểu diễn user và item trong không gian latent factor, trong đó score tương tác được tính từ tích vô hướng giữa vector user và vector item, có thể kết hợp thêm bias user/item [5]. Project sử dụng MF như baseline học máy chính vì mô hình này đơn giản, dễ kiểm soát, có khả năng hấp thụ cả preference cá nhân và popularity item thông qua bias.

2.3 Neural Collaborative Filtering

Neural Collaborative Filtering được He và cộng sự đề xuất nhằm thay thế phép inner product cố định trong MF bằng một kiến trúc neural có khả năng học hàm tương tác user-item linh hoạt hơn [4]. Mô hình NCF thường kết hợp nhánh GMF và MLP để vừa giữ lại cấu trúc collaborative filtering tuyến tính, vừa học quan hệ phi tuyến. Tuy nhiên, việc mô hình neural phức tạp hơn không đảm bảo luôn thắng baseline đơn giản; Dacrema và cộng sự chỉ ra rằng trong nhiều nghiên cứu neural recommender, một số mô hình phức tạp có thể bị vượt bởi baseline được tune tốt [11]. Vì vậy trong báo cáo này, kết quả NCF thấp hơn MF được xem là kết quả thực nghiệm hợp lý, không mâu thuẫn với lý thuyết.

2.4 Visual feature trong fashion recommendation

Với sản phẩm thời trang, hình ảnh chứa thông tin quan trọng về màu sắc, dáng, chất liệu trực quan và phong cách. He và McAuley trong VBPR cho thấy visual appearance có thể được đưa vào personalized ranking bằng cách dùng feature ảnh trích xuất từ deep network [7]. Project này dùng ResNet-50 pretrained làm backbone trích xuất visual feature. ResNet được He và cộng sự đề xuất với residual connection để huấn luyện mạng sâu hiệu quả hơn [6]. Việc sử dụng ResNet pretrained là lựa chọn thực dụng: không cần fine-tune một CNN lớn từ đầu nhưng vẫn có vector ảnh giàu thông tin.

2.5 Đánh giá trong fashion recommendation

Kaggle H&M dùng MAP@12 làm metric chính cho bài toán gợi ý top-12 sản phẩm [1]. Ngoài metric tổng thể, Sherman và cộng sự nhấn mạnh rằng fashion recommendation nên được đánh giá theo nhiều segment user khác nhau, vì domain thời trang có nhiều user mới và hành vi thay đổi mạnh theo ngữ cảnh [10]. Do đó báo cáo không chỉ trình bày bảng overall metric mà còn phân tích theo lịch sử user, độ phổ biến item và độ phủ visual feature.

3 Mô tả bài toán

3.1 Đầu vào và đầu ra

Với mỗi khách hàng u , hệ thống cần trả về danh sách 12 sản phẩm:

$$R_u = [i_1, i_2, \dots, i_{12}] \quad (1)$$

trong đó item đứng trước được xem là có mức độ phù hợp cao hơn item đứng sau. Đầu vào gồm:

- Lịch sử giao dịch: khách hàng, sản phẩm, ngày mua, giá, kênh bán.

- Thông tin sản phẩm: loại hàng, nhóm hàng, màu sắc, chỉ mục mô tả sản phẩm.
- Thông tin khách hàng: tuổi, trạng thái thành viên, khu vực.
- Ảnh sản phẩm: dùng để trích xuất visual embedding.

3.2 Đặc điểm implicit feedback

Dữ liệu mua hàng là implicit feedback. Khi user mua một sản phẩm, ta xem cặp (u, i) là positive. Tuy nhiên, khi user không mua một sản phẩm, không thể kết luận chắc chắn user không thích sản phẩm đó. Đây là điểm khác biệt quan trọng giữa implicit feedback và explicit rating [2]. Vì vậy khi huấn luyện, negative sample được sinh từ các item mà user chưa mua. Với mỗi positive, pipeline dùng 4 negative sample để mô hình học phân biệt sản phẩm phù hợp và không phù hợp.

3.3 Mô hình hóa bài toán xếp hạng

Về mặt toán học, hệ thống cần học một hàm chấm điểm:

$$f_{\theta}(u, i) \rightarrow s_{ui} \quad (2)$$

trong đó u là khách hàng, i là sản phẩm, θ là tham số mô hình và s_{ui} là điểm phù hợp giữa khách hàng và sản phẩm. Với mỗi user, toàn bộ candidate item được sắp xếp giảm dần theo s_{ui} , sau đó lấy 12 item đầu tiên:

$$R_u^{12} = \text{Top12}_{i \in \mathcal{I}} f_{\theta}(u, i) \quad (3)$$

Khác với bài toán phân loại, hệ thống không chỉ cần biết một cặp user-item là “mua” hay “không mua”. Điều quan trọng hơn là thứ tự tương đối giữa các item. Một item đứng đứng ở vị trí 1 có giá trị hơn nhiều so với đứng ở vị trí 12. Vì vậy các metric xếp hạng như MAP@12 và NDCG@12 phù hợp hơn accuracy thông thường cho recommendation top- N [1, 8].

3.4 Thách thức cold-start và long-tail

Dữ liệu H&M có hai loại khó khăn chính. Thứ nhất là cold-start user: một số khách hàng trong test không có hoặc có rất ít lịch sử ở train, khiến mô hình collaborative filtering khó học embedding cá nhân hóa. Thứ hai là long-tail item: rất nhiều sản phẩm chỉ có ít lượt mua, trong khi một nhóm nhỏ sản phẩm phổ biến chiếm phần lớn tương tác. Các nghiên cứu về fashion recommendation cũng nhấn mạnh rằng cần đánh giá theo nhóm user/item thay vì chỉ nhìn metric tổng thể [10]. Đây là động lực để thử mô hình Hybrid có thêm metadata và ảnh sản phẩm.

4 Dữ liệu và tiền xử lý

4.1 Nguồn dữ liệu

Bộ dữ liệu gốc gồm các bảng chính:

Bảng 1: Các thành phần dữ liệu sử dụng

Thành phần	Vai trò
Transactions	Lịch sử mua hàng, là nguồn chính để tạo positive interaction.
Articles	Metadata của sản phẩm như loại sản phẩm, nhóm hàng, màu sắc, mô tả.
Customers	Metadata của khách hàng như tuổi, trạng thái thành viên, khu vực.
Images	Ảnh sản phẩm, dùng để trích xuất đặc trưng thị giác bằng CNN pretrained.

4.2 Tiền xử lý và split theo thời gian

Sau khi tải dữ liệu từ Kaggle, pipeline thực hiện các bước:

1. Đọc các file CSV gốc.
2. Mã hóa `customer_id` thành `user_id` và `article_id` thành `item_id`.
3. Sắp xếp giao dịch theo thời gian.
4. Giữ 15 tháng gần nhất để giảm nhiễu từ hành vi quá cũ.
5. Chia train/test theo mốc thời gian 2020-09-15.
6. Lưu dữ liệu đã xử lý thành các file parquet.

Khoảng thời gian dữ liệu được thể hiện trong Bảng 2.

Bảng 2: Khoảng thời gian train/test

Tập dữ liệu	Bắt đầu	Kết thúc
Dữ liệu gốc	2018-09-20	2020-09-22
Dữ liệu sau lọc 15 tháng	2019-06-22	2020-09-22
Train	2019-06-22	2020-09-14
Test	2020-09-15	2020-09-22

Việc split theo thời gian quan trọng vì hệ thống gợi ý thực tế luôn dự đoán tương lai từ dữ liệu quá khứ. Nếu dùng random split, mô hình có thể nhìn thấy các tín hiệu không thực tế, làm kết quả offline cao hơn nhưng kém tin cậy.

4.3 Lý do mã hóa ID và chuẩn hóa bảng dữ liệu

Các mô hình embedding trong PyTorch cần chỉ số nguyên liên tục để truy cập hàng trong embedding table. Vì vậy `customer_id` gốc được mã hóa thành `user_id` và `article_id` gốc được mã hóa thành `item_id`. Nếu không mã hóa, ID dạng chuỗi dài hoặc ID không liên tục sẽ làm embedding table khó xây dựng và gây lãng phí bộ nhớ.

Quá trình chuẩn hóa dữ liệu cũng tách rõ ba nhóm thông tin:

- **Interaction table:** chứa quan hệ user-item theo thời gian, là nguồn học collaborative filtering.
- **User table:** chứa đặc trưng khách hàng, có thể dùng cho mô hình mở rộng hoặc phân tích segment.
- **Item table:** chứa metadata sản phẩm, dùng cho Hybrid và phân tích theo nhóm item.

Việc lưu dưới dạng parquet giúp đọc nhanh hơn CSV và giữ schema ổn định cho các lần train sau. Trong pipeline full profile, các mô hình đều dùng cùng tập parquet, giúp tránh sai khác do tiền xử lý không nhất quán.

4.4 Kiểm soát leakage khi chia dữ liệu

Temporal split giúp giảm nguy cơ leakage so với random split. Nếu random split theo dòng giao dịch, một user có thể có giao dịch trong tương lai nằm ở train và giao dịch quá khứ nằm ở test, làm thiết lập đánh giá không còn phản ánh thực tế. Với split theo ngày, tất cả tương tác trong test đều xảy ra sau train. Đây là cách đánh giá bảo thủ hơn cho hệ thống gợi ý theo thời gian.

Tuy nhiên temporal split cũng có hệ quả: test week có thể chịu ảnh hưởng mạnh bởi chiến dịch bán hàng, mùa vụ hoặc item mới. Điều này làm Popularity baseline trở nên mạnh hơn trong một số giai đoạn. Vì vậy báo cáo phân tích thêm mức tập trung popularity và full-catalog filtered baseline để tránh kết luận quá đơn giản từ một bảng metric tổng thể.

4.5 Quy mô dữ liệu

Phiên bản dữ liệu mới có 19,196,530 lượt mua trong train và 266,364 lượt mua trong test. Bảng sản phẩm có 105,542 sản phẩm. Số lượng khách hàng trong bảng customers là 1,371,980. Đây là quy mô đủ lớn để các mô hình embedding học được tín hiệu collaborative filtering, nhưng cũng tạo ra thách thức lớn về tốc độ huấn luyện và đánh giá.

4.6 Độ phủ train/test

Trong tập test, có 67,973 user cũng xuất hiện trong train, chiếm khoảng 90.05% test users. Có 7,508 user test không xuất hiện trong train. Về item, có 17,916 item test cũng xuất hiện trong train, chiếm khoảng 95.89% test items; 768 item test không xuất hiện trong train.

Điều này có hai ý nghĩa. Thứ nhất, phần lớn test vẫn có thể được mô hình collaborative filtering học từ lịch sử. Thứ hai, vẫn tồn tại cold-start user và item, khiến mô hình chỉ dựa vào embedding lịch sử gặp khó khăn. Đây là lý do nhóm thử mô hình Hybrid có metadata và visual feature.

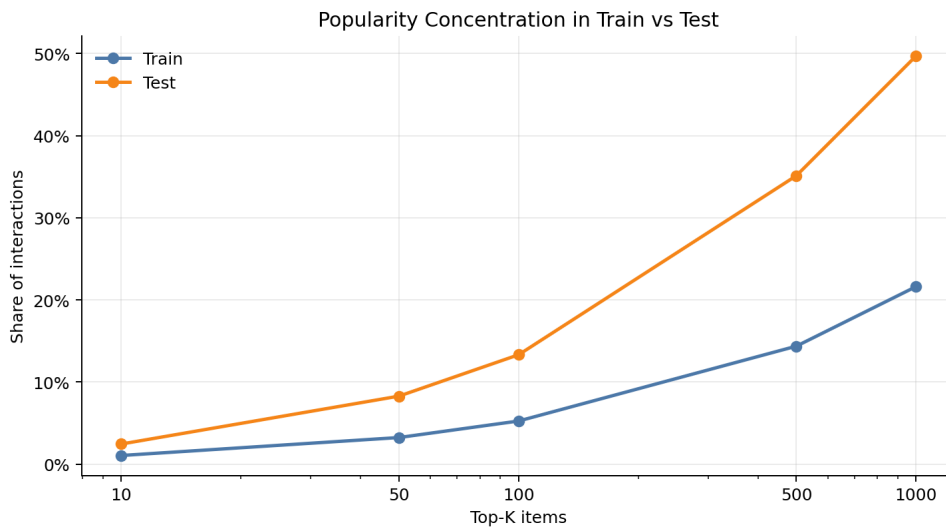
4.7 Tính tập trung theo popularity

Dữ liệu H&M có tính tập trung cao ở tuần test. Bảng 3 cho thấy tỷ trọng tương tác thuộc nhóm item phổ biến nhất.

Bảng 3: Tỷ trọng tương tác của các nhóm item phổ biến

Nhóm item	Train share	Test share
Top 10	0.0104	0.0244
Top 50	0.0323	0.0828
Top 100	0.0525	0.1332
Top 500	0.1436	0.3513
Top 1000	0.2161	0.4970

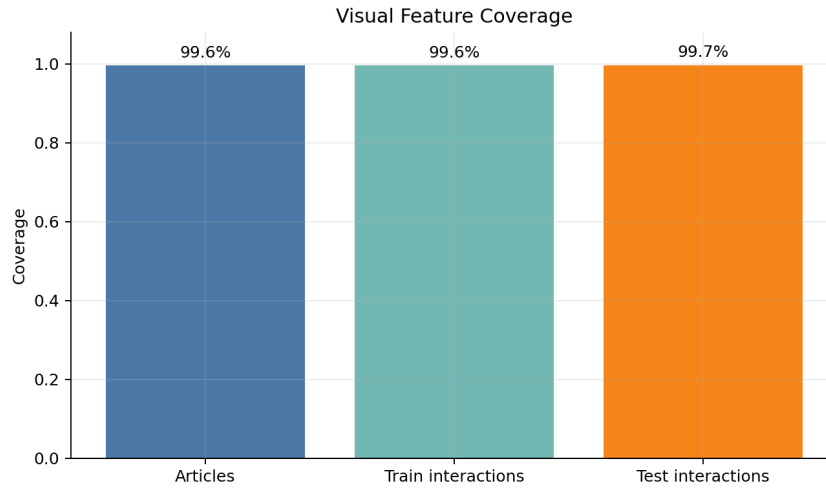
Top 1000 item chiếm gần 49.70% tương tác test. Đây là dấu hiệu rất mạnh cho thấy popularity và trend ngắn hạn có ảnh hưởng lớn đến metric tổng thể.



Hình 1: Mức tập trung popularity trong train và test

4.8 Visual feature

Ảnh sản phẩm được trích xuất thành vector đặc trưng bằng ResNet-50 pretrained. ResNet sử dụng residual connection để hỗ trợ huấn luyện mạng sâu, và là một backbone phổ biến cho trích xuất đặc trưng ảnh [6]. Sau quá trình chạy trên Kaggle, file `visual_features_full.npz` chứa 105,100 vector, đạt độ phủ 99.58% trên toàn bộ article, 99.65% trên train interactions và 99.70% trên test interactions.



Hình 2: Độ phủ visual feature sau khi trích xuất đầy đủ

Vì độ phủ visual gần 100%, việc Hybrid chưa thắng metric tổng thể không còn có thể giải thích chủ yếu bằng lý do thiếu ảnh. Nguyên nhân hợp lý hơn là tính trend/popularity của dữ liệu, cách negative sampling, kiến trúc Hybrid hiện tại và mismatch giữa loss train với metric ranking.

4.9 Quy trình trích xuất đặc trưng ảnh

Quy trình trích xuất visual feature được tách khỏi training chính để tránh việc mỗi epoch phải đọc lại hàng trăm nghìn ảnh. Cách dùng visual feature pretrained cho recommendation sản phẩm cũng phù hợp với hướng VBPR, trong đó visual appearance được đưa vào mô hình ranking từ implicit feedback [7]. Với mỗi ảnh sản phẩm, pipeline thực hiện các bước:

1. Đọc ảnh từ thư mục images của bộ dữ liệu H&M.
2. Resize và chuẩn hóa ảnh theo preprocessing chuẩn của ImageNet.
3. Đưa ảnh qua ResNet-50 pretrained.
4. Loại bỏ fully-connected classification head ở cuối mạng.
5. Lấy vector sau global average pooling làm embedding ảnh của article.

6. Lưu các vector vào shard, sau đó merge thành `visual_features_full.npz`.

Nếu ký hiệu ảnh của item i là x_i , backbone ResNet là $g(\cdot)$, visual vector được tính:

$$v_i = g(x_i), \quad v_i \in \mathbb{R}^{2048} \quad (4)$$

Cách làm này có hai ưu điểm. Thứ nhất, training MF/NCF/Hybrid không bị nghẽn bởi I/O ảnh. Thứ hai, visual feature có thể tái sử dụng cho nhiều lần thử nghiệm khác nhau. Hạn chế là ResNet-50 pretrained trên ImageNet chưa được fine-tune riêng cho domain thời trang, nên embedding có thể chưa tối ưu cho các khác biệt tinh tế như form dáng, chất liệu hoặc phong cách phối đồ. Đây là lý do phần hướng phát triển đề xuất fine-tune visual encoder hoặc học thêm projection layer chuyên biệt cho thời trang, thay vì coi vector ResNet gốc là biểu diễn tối ưu tuyệt đối.

5 Cơ sở lý thuyết

5.1 MAP@12

Metric chính là MAP@12. Với user u , gọi G_u là tập item user thật sự mua trong test, R_u^{12} là danh sách 12 item mô hình đề xuất, và $rel_u(k)$ bằng 1 nếu item ở vị trí k thuộc G_u .

Average Precision@12 được tính:

$$AP@12(u) = \frac{1}{\min(|G_u|, 12)} \sum_{k=1}^{12} P@k(u) \cdot rel_u(k) \quad (5)$$

MAP@12 là trung bình trên toàn bộ user:

$$MAP@12 = \frac{1}{|U|} \sum_{u \in U} AP@12(u) \quad (6)$$

Metric này thưởng cho việc đưa item đúng lên vị trí cao trong top-12, phù hợp hơn accuracy thông thường cho bài toán recommendation. Ví dụ, nếu một item đúng xuất hiện ở vị trí 1, precision tại thời điểm hit cao hơn nhiều so với khi item đúng xuất hiện ở vị trí 12. Vì vậy AP@12 không chỉ kiểm tra có hit hay không, mà còn xét vị trí xuất hiện của hit trong danh sách. Kaggle H&M dùng MAP@12 làm metric chính để phản ánh mục tiêu top-12 ranking [1].

5.2 HitRate@12 và NDCG@12

HitRate@12 đo xem top-12 có ít nhất một item đúng hay không:

$$HR@12 = \frac{1}{|U|} \sum_{u \in U} \mathbf{1}(R_u^{12} \cap G_u \neq \emptyset) \quad (7)$$

NDCG@12 đo chất lượng thứ tự xếp hạng với discount theo vị trí. Ý tưởng DCG/NDCG được dùng rộng rãi trong Information Retrieval để đánh giá danh sách xếp hạng khi các vị trí đầu quan trọng hơn vị trí sau [8]:

$$DCG@12(u) = \sum_{k=1}^{12} \frac{rel_u(k)}{\log_2(k+1)} \quad (8)$$

$$NDCG@12(u) = \frac{DCG@12(u)}{IDCG@12(u)} \quad (9)$$

Trong báo cáo này, MAP@12 là metric chính, còn HitRate@12 và NDCG@12 được dùng để kiểm tra thêm khả năng đánh trúng và chất lượng thứ tự. HitRate dễ hiểu nhưng thô hơn MAP/NDCG vì chỉ cần top-12 có một item đúng là được tính hit, không phân biệt item đúng đứng đầu hay đứng cuối.

5.3 Binary Cross Entropy và negative sampling

Các mô hình học sâu được huấn luyện trên mẫu (u, i, y) với $y \in \{0, 1\}$. Mô hình dự đoán score s_{ui} , sau đó đưa qua sigmoid:

$$\hat{y}_{ui} = \sigma(s_{ui}) \quad (10)$$

Loss sử dụng là Binary Cross Entropy:

$$\mathcal{L}_{BCE} = -y \log(\hat{y}_{ui}) - (1 - y) \log(1 - \hat{y}_{ui}) \quad (11)$$

Một điểm cần lưu ý là BCE tối ưu khả năng phân biệt positive/negative ở cấp cặp user-item, trong khi MAP@12 đánh giá xếp hạng top-12. Vì vậy loss giảm không đảm bảo MAP tăng. Đây là khác biệt giữa pointwise learning objective và ranking metric, cũng là lý do các phương pháp như BPR được đề xuất để tối ưu trực tiếp thứ tự tương đối giữa positive và negative item [3].

5.4 Pairwise ranking và listwise ranking

BCE là pointwise loss vì nó xét từng cặp (u, i) độc lập. Trong recommendation, một cách tiếp cận khác là pairwise ranking: mô hình học để score của item dương lớn hơn score của item âm đối với cùng user. Một loss phổ biến là Bayesian Personalized Ranking (BPR):

$$\mathcal{L}_{BPR} = - \sum_{(u, i^+, i^-)} \log \sigma(s_{ui^+} - s_{ui^-}) \quad (12)$$

Trong đó i^+ là item user đã mua và i^- là item negative. Loss này gần với mục tiêu ranking hơn BCE vì trực tiếp tối ưu thứ tự tương đối giữa item đúng và item sai [3].

Listwise loss đi xa hơn khi xét cả danh sách item cùng lúc. Các loss dạng này thường phù hợp với metric như MAP hoặc NDCG hơn về mặt mục tiêu tối ưu, nhưng phức tạp và tốn tài nguyên hơn khi số lượng item lớn. Trong phạm vi project, nhóm dùng BCE để giữ pipeline ổn định và dễ so sánh giữa MF, NCF và Hybrid; phần hướng phát triển đề xuất thử BPR/listwise để cải thiện ranking.

5.5 Sampled candidate và full-catalog retrieval

Trong bài toán thực tế, mỗi user có thể được xếp hạng trên toàn bộ catalog $\mathcal{I}$ gồm hàng chục nghìn item. Đây gọi là full-catalog retrieval:

$$R_u^{12} = \text{Top12}_{i \in \mathcal{I}} s_{ui} \quad (13)$$

Tuy nhiên full-catalog evaluation tốn nhiều chi phí. Pipeline hiện tại dùng sampled candidate: với mỗi user, item đúng được đánh giá cùng một tập negative candidate được lấy mẫu. Cách này nhanh hơn và phù hợp để so sánh trong quá trình phát triển, nhưng có thể làm sai lệch độ mạnh tương đối của một số baseline, đặc biệt là Popularity. Vì vậy báo cáo có thêm chẩn đoán full-catalog filtered cho Popularity để tránh diễn giải quá mức.

6 Các mô hình

6.1 Popularity baseline

Popularity baseline đếm số lần mỗi item xuất hiện trong train:

$$\text{score}(i) = \text{count}_{\text{train}}(i) \quad (14)$$

Sau đó mô hình đề xuất các item có tần suất mua cao nhất. Baseline này không cá nhân hóa theo user, nhưng trong các bài toán thời trang ngắn hạn, nó có thể rất mạnh vì phản ánh xu hướng phổ biến trong giai đoạn gần test. Các nghiên cứu về đánh giá recommender cũng thường dùng most-popular strategy như một baseline quan trọng, đặc biệt khi phân tích theo segment user [10, 9].

Trong project, Popularity được dùng như mốc so sánh tối thiểu nhưng không hề là baseline yếu. Với dữ liệu có tính xu hướng mạnh, Popularity có thể tận dụng trực tiếp tín hiệu “sản phẩm đang bán chạy”. Hạn chế của mô hình này là mọi user nhận danh sách gần như giống nhau, không khai thác sở thích cá nhân, không xử lý tốt nhu cầu đặc thù và không hiểu nội dung sản phẩm.

6.2 Matrix Factorization

Matrix Factorization học embedding ẩn cho user và item, đúng với cách tiếp cận latent factor được trình bày trong Koren, Bell và Volinsky [5]:

$$s_{ui} = p_u^\top q_i + b_u + b_i \quad (15)$$

trong đó p_u là vector ẩn của user, q_i là vector ẩn của item, b_u và b_i là bias. MF đơn giản hơn NCF và Hybrid, nhưng có ưu điểm là ổn định, ít tham số và học trực tiếp từ lịch sử tương tác.

Trong project, MF đóng vai trò baseline học máy/collaborative filtering quan trọng. Nếu một mô hình phức tạp hơn không vượt được MF, cần xem lại mức overfitting, loss hoặc cách kết hợp feature. Nhận định này cũng phù hợp với các phân tích reproducibility trong recommender systems, nơi baseline đơn giản nhưng được tune tốt có thể cạnh tranh mạnh với neural model phức tạp [11].

Quy trình xây dựng MF gồm các bước:

1. Tạo embedding table cho user: $P \in \mathbb{R}^{|\mathcal{U}| \times d}$.
2. Tạo embedding table cho item: $Q \in \mathbb{R}^{|\mathcal{I}| \times d}$.
3. Với mỗi batch (u, i, y) , lấy p_u và q_i từ embedding table.
4. Tính score bằng tích vô hướng cộng bias.
5. Đưa score qua sigmoid để nhận xác suất tương tác.
6. Cập nhật embedding bằng backpropagation theo BCE loss.

Bias b_i có vai trò quan trọng vì nó học độ phổ biến riêng của item. Bias b_u học xu hướng hoạt động của user. Do đó MF không chỉ học sở thích ẩn mà còn hấp thụ một phần popularity signal. Đây là một lý do MF có kết quả khá tốt trên dữ liệu H&M.

6.3 Neural Collaborative Filtering

NCF thay phép nhân tuyến tính của MF bằng mạng neural để học tương tác phi tuyến giữa user và item [4]. Một cách biểu diễn đơn giản:

$$h_{ui} = MLP([p_u, q_i]) \quad (16)$$

$$s_{ui} = w^\top h_{ui} + b \quad (17)$$

Trong kiến trúc NeuMF thường có hai nhánh:

- GMF branch: nhân từng chiều embedding user/item.
- MLP branch: ghép embedding rồi đưa qua các dense layer.

NCF có khả năng biểu diễn tốt hơn MF, nhưng cũng dễ overfit hơn nếu dữ liệu, negative sampling hoặc regularization chưa phù hợp.

Trong project, NCF được dùng để kiểm tra giả thuyết: quan hệ user-item trong H&M có cần mô hình phi tuyến hay không. Kiến trúc NCF gồm các embedding riêng cho GMF branch và MLP branch như gợi ý trong NeuMF [4]. GMF giữ lại ý tưởng nhân từng chiều giống MF:

$$h_{GMF} = p_u^G \odot q_i^G \quad (18)$$

MLP branch ghép embedding rồi đưa qua nhiều lớp dense:

$$h_{MLP} = MLP([p_u^M; q_i^M]) \quad (19)$$

Vector cuối cùng được ghép lại:

$$h_{NCF} = [h_{GMF}; h_{MLP}] \quad (20)$$

$$s_{ui} = w^\top h_{NCF} + b \quad (21)$$

Ưu điểm của NCF là có thể học các quan hệ phi tuyến, ví dụ một user chỉ thích một nhóm item khi chúng đồng thời có nhiều đặc điểm ngầm phù hợp. Nhược điểm là số tham số lớn hơn, yêu cầu regularization tốt hơn và dễ học nhiễu từ negative sample hơn MF.

6.4 Hybrid đa phương thức

Hybrid trong project kết hợp ba nhóm tín hiệu. Việc đưa visual signal vào recommendation sản phẩm có cơ sở từ hướng visual personalized ranking như VBPR [7]:

- Latent user-item từ NCF.
- Metadata sản phẩm.
- Visual embedding từ ResNet-50.

Dạng tổng quát:

$$z_{ui} = [h_{ui}^{NCF}; m_i; v_i] \quad (22)$$

$$s_{ui} = MLP_{hybrid}(z_{ui}) \quad (23)$$

Trong đó m_i là metadata của item và v_i là visual vector. Kỳ vọng của Hybrid là tận dụng tín hiệu nội dung để hỗ trợ các item ít lịch sử hoặc item mới, điều mà MF/NCF thuần collaborative filtering khó xử lý.

Quy trình xây dựng Hybrid trong project:

1. Dùng NCF backbone để tạo latent representation cho cặp user-item.
2. Chuẩn bị metadata item từ bảng articles, ví dụ nhóm hàng, loại sản phẩm, màu sắc.
3. Load visual vector v_i từ file NPZ đã trích xuất trước.
4. Ghép các tín hiệu thành vector chung z_{ui} .
5. Đưa z_{ui} qua MLP để dự đoán score.

Một dạng cụ thể của tầng dự đoán:

$$a_1 = ReLU(W_1 z_{ui} + b_1) \quad (24)$$

$$a_2 = ReLU(W_2 a_1 + b_2) \quad (25)$$

$$s_{ui} = W_3 a_2 + b_3 \quad (26)$$

Khi item thiếu visual feature, pipeline có thể thay bằng vector 0 hoặc bỏ qua tín hiệu ảnh. Trong phiên bản dữ liệu mới, độ phủ ảnh gần 100%, nên phần lớn item đều có visual vector. Về mặt thiết kế, Hybrid phù hợp hơn MF/NCF cho long-tail vì metadata và ảnh không phụ thuộc hoàn toàn vào số lượt mua trong train. Tuy nhiên, đây là kỳ vọng về mặt kiến trúc; hiệu quả thực tế vẫn phải được kiểm chứng bằng metric và segment analysis.

6.5 So sánh mức độ thông tin của các mô hình

Bảng 4: So sánh tín hiệu đầu vào của các mô hình

Mô hình	User ID	Item ID	Metadata	Ảnh
Popularity	Không	Có	Không	Không
MF	Có	Có	Không	Không
NCF	Có	Có	Không	Không
Hybrid	Có	Có	Có	Có

Bảng 4 cho thấy Hybrid giàu thông tin nhất, nhưng giàu thông tin không đảm bảo metric cao nhất. Khi dữ liệu test bị chi phối mạnh bởi head item, một baseline đơn giản nhưng khớp với xu hướng như Popularity vẫn có thể thắng. Điều này nhấn mạnh vai trò của đánh giá thực nghiệm thay vì chỉ đánh giá mô hình theo độ phức tạp kiến trúc.

Bảng 5: Ưu điểm và hạn chế của các mô hình trong phạm vi project

Mô hình	Ưu điểm	Hạn chế
Popularity	Rất đơn giản, ổn định, phản ánh tốt trend ngắn hạn; là baseline quan trọng trong đánh giá recommender [10, 9].	Không cá nhân hóa; không dùng lịch sử riêng của user; không hiểu metadata/ảnh; yếu với nhu cầu đặc thù.
MF	Học latent factor user/item, ít tham số hơn neural model, dễ kiểm soát và thường là baseline mạnh [5].	Khó xử lý item mới hoặc item ít tương tác; quan hệ user-item chủ yếu tuyến tính qua dot product.
NCF	Có khả năng học hàm tương tác phi tuyến giữa user và item; tổng quát hóa ý tưởng MF trong một framework neural [4].	Nhiều tham số hơn, dễ overfit hơn nếu dữ liệu/regularization chưa đủ; loss pointwise có thể lệch với metric ranking.
Hybrid	Kết hợp collaborative signal với metadata và visual feature; phù hợp hơn về mặt thiết kế cho item ít lịch sử và item giàu nội dung [7].	Kiến trúc concat đơn giản chưa chắc tận dụng tốt mọi tín hiệu; visual encoder chưa fine-tune theo domain thời trang.

7 Pipeline thực nghiệm

7.1 Lệnh chạy chính

Thực nghiệm full profile được chạy bằng lệnh:

```
python scripts\run_report_pipeline.py --profile full --stages check-data,train-hybrid,train-compare,test --data-dir data\processed_new --visual-features data\processed_new\visual_features_full.npz --checkpoint-dir checkpoints\processed_new --output-root reports\report_pipeline_processed_new --models popularity,mf,ncf,hybrid
```

Các stage chính gồm:

- **check-data**: kiểm tra dữ liệu, overlap, độ phủ visual feature.
- **train-hybrid**: train riêng Hybrid và lưu best checkpoint.
- **train-compare**: train/compare các mô hình còn lại.
- **test**: load checkpoint tốt nhất và đánh giá trên tập test.

7.2 Luồng xây dựng mô hình trong code

Pipeline được thiết kế để các mô hình dùng chung một chuẩn dữ liệu và chuẩn đánh giá. Luồng xử lý tổng quát:

1. **Load dữ liệu parquet:** đọc train/test interactions, articles và customers.
2. **Kiểm tra schema:** đảm bảo các cột bắt buộc như `user_id`, `item_id`, `t_dat` tồn tại.
3. **Tạo dataset implicit feedback:** giữ positive interaction từ giao dịch thật và sinh negative sample cho từng positive.
4. **Tạo DataLoader:** batch hóa dữ liệu, shuffle train và dùng nhiều worker để tăng tốc đọc batch.
5. **Khởi tạo model:** tùy model sẽ tạo embedding table, MLP hoặc load visual feature.
6. **Train từng epoch:** forward, tính BCE loss, backward, cập nhật tham số bằng optimizer.
7. **Validation:** đo validation loss sau mỗi epoch.
8. **Checkpoint:** lưu model tốt nhất theo validation loss.
9. **Test:** load best checkpoint và sinh top-12 recommendation.
10. **Tính metric:** MAP@12, HitRate@12 và NDCG@12.

Thiết kế này giúp so sánh công bằng hơn giữa MF, NCF và Hybrid vì các mô hình dùng chung split, negative sampling, metric và logic checkpoint.

7.3 Chi tiết negative sampling trong training

Với mỗi interaction dương (u, i^+) , pipeline lấy mẫu một số item i^- mà user u chưa mua trong train. Batch huấn luyện vì vậy gồm cả positive và negative:

$$\mathcal{B} = \{(u, i^+, 1)\} \cup \{(u, i_1^-, 0), \dots, (u, i_K^-, 0)\} \quad (27)$$

với $K = 4$ trong full profile.

Negative sampling giúp giảm chi phí huấn luyện. Nếu không lấy mẫu, với mỗi user cần so sánh trên toàn bộ catalog sản phẩm, tạo ra số cặp user-item rất lớn. Cách làm này là phổ biến trong implicit feedback recommendation, nhưng cũng có hạn chế: negative sample không nhất thiết là sản phẩm user không thích, mà chỉ là sản phẩm chưa quan sát thấy user mua [2, 3]. Vì vậy chất lượng negative sampling ảnh hưởng trực tiếp tới khả năng ranking của mô hình.

7.4 Batch training và tối ưu

Trong mỗi epoch, DataLoader sinh các mini-batch. Với mỗi batch, model thực hiện:

1. Forward pass để tính score s_{ui} .
2. Đưa score qua sigmoid để nhận $\hat{y}_{ui}$.
3. Tính BCE loss trên batch.
4. Backward pass để tính gradient.
5. Optimizer cập nhật embedding và trọng số MLP.

MF chủ yếu cập nhật embedding table và bias. NCF cập nhật thêm các dense layer trong GMF/MLP branch. Hybrid cập nhật backbone user-item và MLP kết hợp feature. Do Hybrid có thêm visual/metadata input, chi phí xử lý mỗi batch cao hơn MF. Vì vậy pipeline đã bật nhiều worker cho MF/NCF giống Hybrid để so sánh tốc độ và kết quả công bằng hơn.

7.5 Thiết lập huấn luyện

Full profile sử dụng toàn bộ train set, 5 epoch cho MF/NCF/Hybrid, 4 negative cho mỗi positive và 1000 negative candidate mỗi user khi đánh giá sampled candidate. Các mô hình học sâu đều dùng cơ chế data loading nhiều worker để tránh thiên vị tốc độ cho Hybrid.

Checkpoint tốt nhất được chọn theo validation loss. Điều này quan trọng vì log huấn luyện cho thấy train loss thường giảm qua epoch, nhưng validation loss tăng sau epoch đầu, biểu hiện overfitting. Pipeline không dùng checkpoint cuối nếu checkpoint đó kém hơn trên validation.

7.6 Best checkpoint và overfitting

Trong recommendation với negative sampling, train loss có thể tiếp tục giảm khi mô hình học ngày càng tốt các negative sample trong train. Tuy nhiên điều này không đồng nghĩa mô hình tổng quát hóa tốt trên test week. Vì vậy pipeline lưu checkpoint theo validation loss:

$$\theta^* = \arg \min_{\theta_e} \mathcal{L}_{val}(\theta_e) \quad (28)$$

trong đó θ_e là tham số model ở epoch e .

Sau khi train xong, pipeline load θ^* để đánh giá test. Cách này tránh việc dùng epoch cuối bị overfit. Với kết quả hiện tại, nhiều mô hình có validation loss tốt nhất ở epoch đầu, cho thấy cần thêm regularization hoặc objective ranking tốt hơn nếu muốn train lâu hơn.

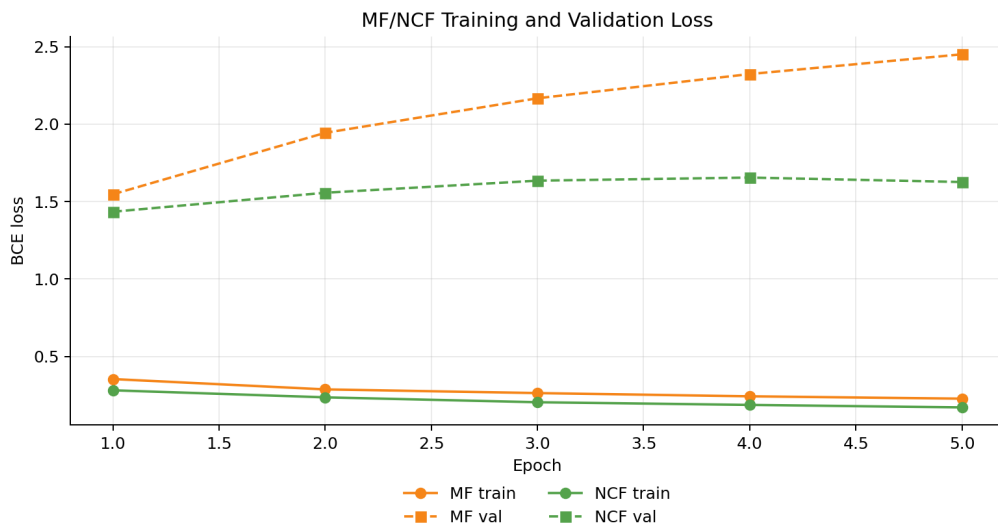
7.7 Cách diễn giải kết quả offline

Kết quả trong báo cáo là offline evaluation. Offline evaluation hữu ích vì có thể chạy lặp lại, so sánh nhiều mô hình và phân tích lỗi trước khi triển khai. Tuy nhiên, các nghiên cứu về recommender systems nhấn mạnh rằng kết quả offline cần được diễn giải cẩn thận, đặc biệt khi candidate set, split thời gian hoặc baseline tuning thay đổi [11]. Vì vậy báo cáo dùng hai lớp phân tích:

- **Overall metrics:** cho biết mô hình nào tốt nhất trên thiết lập test hiện tại.
- **Diagnostics/segments:** kiểm tra mô hình mạnh ở nhóm user/item nào, tránh kết luận chỉ dựa vào một số tổng thể.

Với fashion recommendation, phân tích theo segment càng quan trọng vì user mới, sản phẩm mới và xu hướng thời trang có thể ảnh hưởng rất khác nhau tới từng nhóm [10].

7.8 Biểu đồ training



Hình 3: Train/validation loss trong quá trình huấn luyện

Hình 3 cho thấy loss train giảm nhưng validation loss tăng ở MF/NCF sau các epoch đầu. Với Hybrid, log cũng cho thấy checkpoint tốt nhất nằm ở epoch đầu. Điều này giải thích vì sao cần best checkpoint và vì sao loss thấp hơn không đồng nghĩa mô hình có MAP@12 cao hơn.

8 Tái lập thực nghiệm và artifact kiểm chứng

8.1 Các artifact chính

Để đảm bảo kết quả có thể kiểm tra lại, pipeline lưu riêng dữ liệu, checkpoint, log và báo cáo chẩn đoán. Bảng 6 mô tả các artifact chính.

Bảng 6: Artifact chính của thực nghiệm

Artifact	Vai trò
<code>data\processed_new</code>	Dữ liệu parquet đã xử lý lại, gồm train/test interactions, articles và customers.
<code>visual_features_full.npz</code>	Visual embedding đã trích xuất từ ảnh sản phẩm, dùng cho Hybrid.
<code>checkpoints processed_new</code>	Best checkpoint của các mô hình học sâu.
<code>metrics_comparison.csv</code>	Bảng metric tổng thể MAP@12, HitRate@12, NDCG@12.
<code>diagnostics_summary.md</code>	Tóm tắt độ phủ visual, full-catalog baseline và danh sách biểu đồ sinh ra.
<code>segment_metrics.csv</code>	Metric theo nhóm user history, item popularity và visual availability.

8.2 Checklist tái lập

Một lần chạy full hợp lệ cần thỏa mãn các điều kiện sau:

1. Dữ liệu train/test được split theo thời gian, không random split.
2. File visual feature được truyền rõ qua tham số `-visual-features`.
3. Checkpoint mới được ghi vào thư mục riêng, tránh trộn với checkpoint cũ.
4. Stage test load best checkpoint thay vì checkpoint cuối cùng.
5. Các mô hình được so sánh trên cùng candidate sampling và cùng metric.
6. Báo cáo diagnostics được chạy sau test để kiểm tra segment và độ phủ feature.

Checklist này quan trọng vì chỉ cần thay đổi một trong các điều kiện trên, metric có thể thay đổi và không còn so sánh trực tiếp được với bảng kết quả trong báo cáo.

8.3 Các lệnh đã dùng

Lệnh chạy full pipeline:

```
python scripts\run_report_pipeline.py --profile full --stages check-data,train
-hybrid,train-compare,test --data-dir data\processed_new --visual-features
data\processed_new\visual_features_full.npz --checkpoint-dir checkpoints\
processed_new --output-root reports\report_pipeline_processed_new --models
popularity,mf,ncf,hybrid
```

Sau khi chạy xong, diagnostics được dùng để tạo biểu đồ và các bảng segment. Các hình trong báo cáo này được lấy từ thư mục diagnostics của output root `processed_new`. Việc tách output theo thư mục giúp tránh nhầm kết quả cũ với kết quả mới.

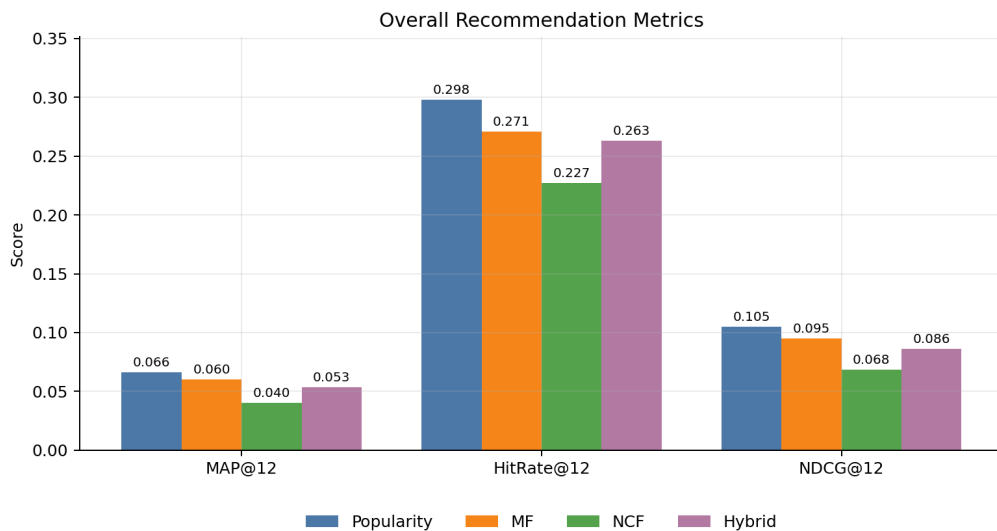
9 Kết quả thực nghiệm

9.1 Kết quả tổng thể

Bảng 7 là kết quả test cuối cùng.

Bảng 7: Kết quả tổng thể trên tập test

Model	MAP@12	HitRate@12	NDCG@12
Popularity	0.06640	0.29790	0.10483
MF	0.06016	0.27097	0.09478
Hybrid	0.05325	0.26317	0.08639
NCF	0.03997	0.22702	0.06816



Hình 4: So sánh MAP@12, HitRate@12 và NDCG@12

Thứ tự tổng thể là:

$$Popularity > MF > Hybrid > NCF \quad (29)$$

Kết quả này cần được trình bày trung thực: Hybrid là mô hình giàu feature hơn, nhưng chưa phải mô hình có metric tổng thể tốt nhất trong pipeline hiện tại.

9.2 Đọc kết quả tổng thể

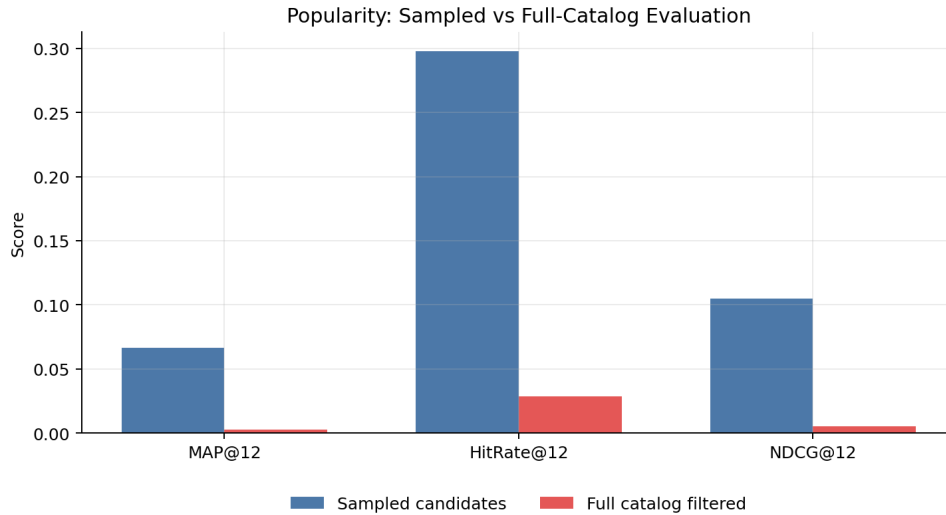
Kết quả tổng thể cho thấy mô hình đơn giản nhất lại đứng đầu. Điều này không bất thường trong recommender systems. Nếu test window chịu ảnh hưởng mạnh bởi trend, Popularity có thể khớp với hành vi mua hàng ngắn hạn tốt hơn mô hình cá nhân hóa. Đồng thời, các nghiên cứu reproducibility về neural recommender cũng chỉ ra rằng mô hình phức tạp hơn không tự động vượt các baseline đơn giản nếu baseline được xây dựng và đánh giá cẩn thận [11].

MF đứng thứ hai cho thấy collaborative filtering vẫn học được tín hiệu cá nhân hóa và độ phổ biến item. Việc MF vượt NCF cho thấy phi tuyến hóa bằng MLP chưa đem lại lợi ích trong thiết lập hiện tại. Nguyên nhân có thể đến từ dữ liệu sparse, negative sampling, overfitting hoặc objective BCE không khớp MAP@12. Hybrid vượt NCF nhưng không vượt MF, cho thấy metadata và ảnh có đóng góp, nhưng cách kết hợp hiện tại chưa đủ để thắng mô hình collaborative filtering đơn giản hơn.

9.3 Phân tích Popularity

Popularity thắng tổng thể vì dữ liệu test có tính xu hướng rất mạnh. Trong tuần test, nhiều lượt mua tập trung vào nhóm sản phẩm phổ biến. Với đánh giá sampled candidate, item phổ biến có xác suất được xếp đúng cao hơn, đặc biệt khi nhiều user có ít lịch sử cá nhân hóa. Đây là lý do Popularity được xem là baseline bắt buộc phải so sánh, không nên coi là mô hình “quá đơn giản” để bỏ qua [10, 9].

Tuy nhiên, khi đánh giá Popularity theo full-catalog filtered baseline, MAP@12 chỉ còn 0.002380, HitRate@12 = 0.028630 và NDCG@12 = 0.005473. Điều này cho thấy sampled evaluation có thể làm Popularity trông mạnh hơn so với full retrieval thực tế.



Hình 5: Popularity trong sampled candidate và full-catalog filtered evaluation

9.4 Phân tích MF

MF đạt $\text{MAP@12} = 0.06016$, đứng thứ hai sau Popularity và là mô hình collaborative filtering tốt nhất trong project hiện tại. Mô hình này đơn giản nhưng ổn định, học được cả sở thích user và độ phổ biến item thông qua embedding và bias.

Việc MF vượt NCF và Hybrid cho thấy kiến trúc phức tạp hơn chưa chắc hiệu quả hơn nếu không khớp tốt với dữ liệu, objective và quy trình đánh giá.

9.5 Phân tích NCF

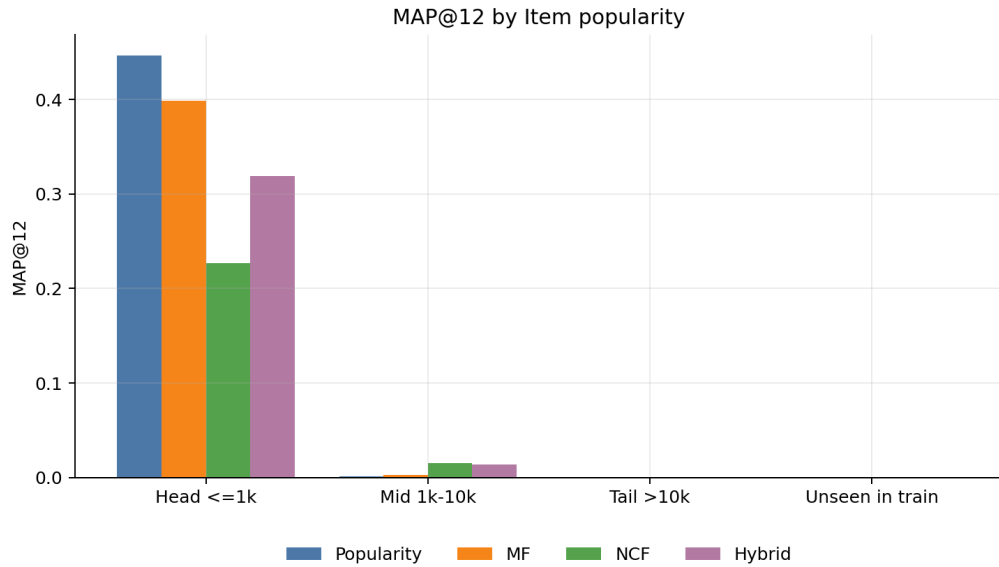
NCF có $\text{MAP@12} = 0.03997$, thấp nhất trong bốn mô hình. Mặc dù NCF có MLP và khả năng biểu diễn phi tuyến, kết quả cho thấy độ phức tạp tăng lên dẫn tới nguy cơ overfit. Ngoài ra, loss BCE trên negative sample chưa chắc tạo ra thứ tự top-12 tốt nhất.

Đây là bài học quan trọng: với recommendation, giảm classification loss không đủ. Mục tiêu thực tế là ranking, vì vậy các hướng như BPR loss, sampled softmax, listwise loss hoặc ranking model tăng hai có thể phù hợp hơn.

9.6 Phân tích Hybrid

Hybrid đạt $\text{MAP@12} = 0.05325$, tốt hơn NCF nhưng thấp hơn MF và Popularity. Điều này cho thấy metadata và ảnh có đóng góp tích cực so với NCF thuần, nhưng cách kết hợp hiện tại chưa đủ mạnh để vượt các baseline tổng thể.

Điểm đáng chú ý là khi phân tích theo nhóm item, Hybrid có tín hiệu tốt hơn Popularity ở một số nhóm mid/tail item. Đây là nơi visual và metadata có ý nghĩa hơn vì item ít lịch sử không được Popularity hỗ trợ tốt. Nhận định này phù hợp với động cơ của các mô hình visual recommendation như VBPR: visual appearance có thể bổ sung tín hiệu khi lịch sử tương tác chưa đủ dày [7].

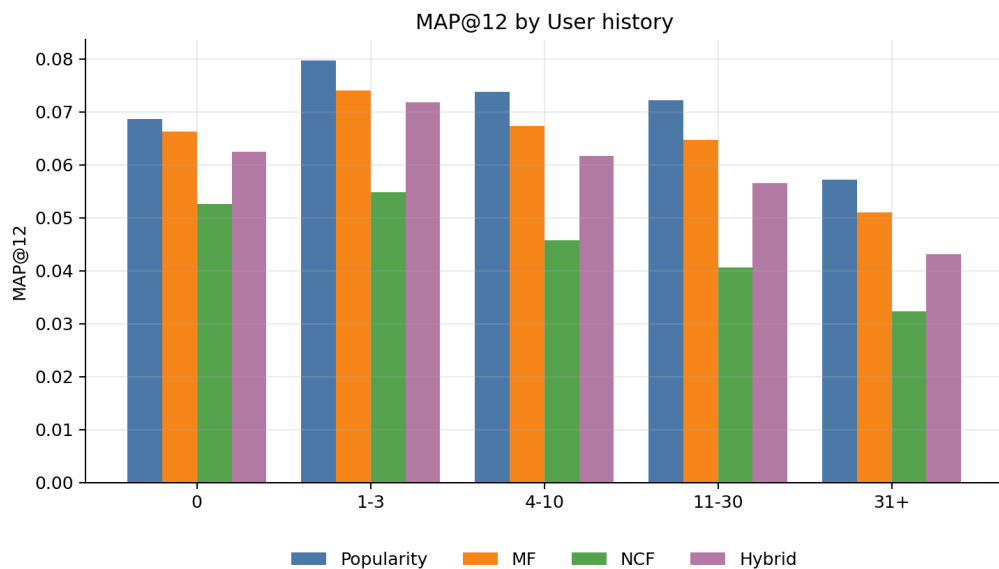


Hình 6: MAP@12 theo nhóm độ phổ biến item

10 Phân tích theo segment

10.1 Theo lịch sử user

Kết quả theo lịch sử user cho thấy user mới hoặc ít lịch sử phụ thuộc nhiều vào xu hướng chung. Với các user có lịch sử dài, hành vi có thể phân tán hơn và khó dự đoán bằng top popular chung.



Hình 7: MAP@12 theo nhóm lịch sử user

Bảng 8: MAP@12 theo nhóm lịch sử user

Nhóm user	Popularity	MF	NCF	Hybrid
0	0.06860	0.06629	0.05252	0.06249
1-3	0.07962	0.07405	0.05482	0.07179
4-10	0.07378	0.06728	0.04580	0.06171
11-30	0.07223	0.06470	0.04065	0.05659
31+	0.05724	0.05098	0.03230	0.04305

10.2 Theo độ phổ biến item

Bảng 9 cho thấy sự khác biệt quan trọng giữa overall metric và segment metric.

Bảng 9: MAP@12 theo nhóm độ phổ biến item

Nhóm item	Popularity	MF	NCF	Hybrid
Head $\leq$ 1k	0.44629	0.39859	0.22700	0.31898
Mid 1k-10k	0.00123	0.00306	0.01522	0.01358
Tail $>$ 10k	0.00000	0.00000	0.00065	0.00083
Unseen in train	0.00000	0.00000	0.00000	0.00014

Ở nhóm head item, Popularity vượt trội. Nhưng ở nhóm mid/tail, NCF và Hybrid có dấu hiệu tốt hơn Popularity. Tuy nhiên giá trị MAP tuyệt đối ở tail rất nhỏ, nên không đủ kéo metric tổng thể vượt lên.

10.3 Theo visual availability

Visual coverage gần như đầy đủ, nhưng nhóm visual missing có số lượng nhỏ nên không nên diễn giải quá mạnh. Kết luận hợp lý là dữ liệu ảnh đã được chuẩn bị đủ tốt để phục vụ Hybrid, nhưng kiến trúc và objective hiện tại chưa tận dụng hết được lợi thế này.

11 Thảo luận

11.1 Mức độ chắc chắn của các kết luận

Báo cáo phân biệt ba loại kết luận:

- Kết luận trực tiếp từ số liệu:** ví dụ Popularity có MAP@12 cao nhất trong bảng test hiện tại. Đây là kết luận từ artifact thực nghiệm của project.
- Diễn giải có căn cứ từ diagnostics:** ví dụ Popularity mạnh do test tập trung vào head item. Diễn giải này dựa trên bảng popularity concentration và segment metrics.

3. **Giả thuyết/hướng cải tiến:** ví dụ BPR hoặc ranker tầng hai có thể cải thiện ranking. Đây là hướng đề xuất dựa trên tài liệu về ranking loss và recommender systems, chưa được chứng minh bằng thực nghiệm trong project này [3, 12, 13].

Cách phân biệt này giúp tránh nhầm lẫn giữa “đã đo được” và “có khả năng cải thiện”. Ví dụ, báo cáo không khẳng định Hybrid chắc chắn tốt hơn nếu fine-tune ResNet; báo cáo chỉ nói đây là hướng hợp lý để thử vì visual signal đã được chứng minh là hữu ích trong một số mô hình visual recommendation [7].

11.2 Vì sao Popularity tốt nhất?

Popularity tốt nhất do ba nguyên nhân chính:

1. Test week của H&M tập trung mạnh vào item phổ biến gần đây.
2. Dữ liệu thời trang có tính mùa vụ và xu hướng ngắn hạn cao.
3. Evaluation sampled candidate làm baseline phổ biến có lợi thế nhất định so với full-catalog retrieval.

Vì vậy kết quả này không có nghĩa Popularity luôn là mô hình tốt nhất trong mọi bối cảnh. Nó chỉ cho thấy trên split và quy trình đánh giá hiện tại, popularity signal là tín hiệu mạnh nhất.

Một cách nhìn khác là Popularity đang đóng vai trò như một mô hình trend-following. Trong thời trang, trend ngắn hạn có thể mạnh hơn preference dài hạn của từng user. Nếu test window trùng với các sản phẩm đang được nhiều người mua, mô hình phổ biến sẽ có lợi thế. Đây là lý do các hệ thống thực tế thường kết hợp popularity, recency và personalization thay vì chỉ dùng một nguồn tín hiệu.

11.3 Vì sao Hybrid chưa thắng?

Hybrid chưa thắng do một số nguyên nhân có thể:

- Kiến trúc hiện tại chủ yếu concat feature, chưa có attention/gating để chọn tín hiệu theo từng user/item.
- Visual feature từ ResNet-50 là feature chung cho ảnh, chưa được fine-tune riêng cho thời trang H&M.
- BCE loss chưa tối ưu trực tiếp MAP@12.
- Candidate generation chưa đủ mạnh; mô hình ranking phải cạnh tranh trong tập candidate sampled thay vì một pipeline retrieval + ranker tối ưu.
- Tín hiệu popularity quá mạnh làm lợi thế nội dung bị che khuất ở metric tổng thể.

Trong đó, các nguyên nhân về architecture và objective là giả thuyết kỹ thuật cần kiểm chứng thêm. Hiện tại số liệu chỉ chứng minh Hybrid chưa thắng overall; số liệu không đủ để kết luận visual feature vô dụng. Ngược lại, segment mid/tail cho thấy tín hiệu nội dung có thể có ích, nhưng mức lợi chưa đủ lớn để kéo MAP tổng thể vượt MF/Popularity.

11.4 Bài học từ thực nghiệm

Thực nghiệm cho thấy mô hình phức tạp hơn không tự động thắng. Trong recommendation, cách split dữ liệu, candidate generation, negative sampling, objective huấn luyện và metric đánh giá ảnh hưởng rất lớn tới kết quả. Một hệ thống thực tế thường không chỉ dùng một neural model end-to-end, mà kết hợp nhiều tầng: candidate generation, feature engineering, ranking và ensemble.

12 Hạn chế

Project hiện tại còn một số hạn chế:

- Đánh giá sampled candidate chưa tương đương leaderboard Kaggle full retrieval.
- Chưa triển khai co-visitation matrix hoặc recent popularity theo tuần ở mức production.
- Chưa có ranking model tầng hai như LightGBM/CatBoost.
- Chưa fine-tune visual encoder trên domain thời trang.
- Chưa tune sâu các siêu tham số như embedding size, negative ratio, learning rate schedule, weight decay và dropout.
- Chưa kiểm thử nhiều split thời gian khác nhau để đánh giá độ ổn định.

13 Hướng phát triển

13.1 Candidate generation

Để tiến gần hơn cách làm hiệu quả trong các cuộc thi recommendation, nên tách hệ thống thành hai tầng. Tầng đầu sinh candidate từ nhiều nguồn:

- Recent popularity theo tuần gần nhất.
- Co-visitation item-item dựa trên lịch sử mua chung.
- User history recall: gợi ý item tương tự với các item user từng mua.
- Content-based recall từ visual similarity và metadata.

13.2 Ranking stage

Sau khi có candidate, dùng ranker để xếp hạng:

- LightGBM hoặc CatBoost ranker. LightGBM là gradient boosting decision tree hiệu quả cho dữ liệu tabular lớn [12]; CatBoost có lợi thế xử lý categorical feature và ordered boosting [13].
- Feature popularity, recency, user profile, item metadata, price, sales channel.
- Feature collaborative score từ MF/NCF/Hybrid.
- Feature visual similarity giữa item candidate và lịch sử mua của user.

Trong hướng này, MF/NCF/Hybrid không nhất thiết phải là mô hình cuối cùng sinh recommendation trực tiếp. Chúng có thể trở thành feature generator: mỗi mô hình cung cấp một score hoặc embedding, sau đó ranker học cách kết hợp với popularity, recency và metadata. Đây thường là cách thực dụng hơn trong hệ thống recommendation quy mô lớn vì tách bài toán recall candidate khỏi bài toán ranking chi tiết.

13.3 Cải tiến Hybrid

Hybrid có thể được cải tiến bằng:

- Gating network để học trọng số giữa collaborative, metadata và visual signal.
- Attention trên lịch sử item của user.
- Fine-tune visual encoder cho domain thời trang.
- Loss hướng ranking như BPR hoặc listwise loss.

14 Kết luận

Project đã xây dựng được pipeline recommendation hoàn chỉnh từ xử lý dữ liệu, trích xuất visual feature, huấn luyện checkpoint, đánh giá mô hình và tạo báo cáo chẩn đoán. Kết quả cuối cùng cho thấy Popularity là mô hình mạnh nhất trên metric tổng thể của split hiện tại, MF là mô hình collaborative filtering ổn định nhất, Hybrid vượt NCF nhưng chưa vượt MF/Popularity.

Kết quả này là hợp lý với đặc thù dữ liệu H&M: test week bị chi phối bởi xu hướng phổ biến và nhiều user có lịch sử hạn chế. Đồng thời, phân tích segment cho thấy Hybrid vẫn có giá trị về mặt nội dung ở các nhóm item ít phổ biến hơn. Hướng phát triển phù hợp là xây dựng pipeline hai tầng gồm candidate generation mạnh và ranking stage có feature đa dạng, thay vì chỉ kỳ vọng một mô hình neural đơn lẻ vượt mọi baseline.

Tài liệu

- [1] Kaggle, *H&M Personalized Fashion Recommendations - Evaluation*, MAP@12 competition metric. [Kaggle evaluation page](#).
- [2] Y. Hu, Y. Koren, C. Volinsky, *Collaborative Filtering for Implicit Feedback Datasets*, IEEE International Conference on Data Mining, 2008. DOI: 10.1109/ICDM.2008.22.
- [3] S. Rendle, C. Freudenthaler, Z. Gantner, L. Schmidt-Thieme, *BPR: Bayesian Personalized Ranking from Implicit Feedback*, UAI 2009. arXiv: <https://arxiv.org/abs/1205.2618>.
- [4] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, T.-S. Chua, *Neural Collaborative Filtering*, WWW 2017. DOI: 10.1145/3038912.3052569. arXiv: <https://arxiv.org/abs/1708.05031>.
- [5] Y. Koren, R. Bell, C. Volinsky, *Matrix Factorization Techniques for Recommender Systems*, IEEE Computer, 2009. DOI: 10.1109/MC.2009.263.
- [6] K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016. DOI: 10.1109/CVPR.2016.90. arXiv: <https://arxiv.org/abs/1512.03385>.
- [7] R. He, J. McAuley, *VBPR: Visual Bayesian Personalized Ranking from Implicit Feedback*, AAAI 2016. arXiv: <https://arxiv.org/abs/1510.01784>.
- [8] K. Järvelin, J. Kekäläinen, *Cumulated Gain-Based Evaluation of IR Techniques*, ACM Transactions on Information Systems, 2002. DOI: 10.1145/582415.582418.
- [9] W. Ji et al., *A Re-visit of the Popularity Baseline in Recommender Systems*, 2020.
- [10] J. Sherman, C. Shukla, R. Textor, S. Zhang, A. A. Winecoff, *Assessing Fashion Recommendations: A Multifaceted Offline Evaluation Approach*, 2019. arXiv: <https://arxiv.org/abs/1909.04496>.
- [11] M. F. Dacrema, S. Boglio, P. Cremonesi, D. Jannach, *A Troubling Analysis of Reproducibility and Progress in Recommender Systems Research*, ACM Transactions on Information Systems, 2021. arXiv: <https://arxiv.org/abs/1911.07698>.
- [12] G. Ke et al., *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*, NeurIPS 2017.
- [13] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, *CatBoost: Unbiased Boosting with Categorical Features*, NeurIPS 2018. arXiv: <https://arxiv.org/abs/1706.09516>.